



L-Università ta' Malta
Faculty of Engineering

UNIVERSITY OF MALTA
Faculty of Engineering

FINAL YEAR PROJECT
B.ENG (HONS)

**Identifying Optimal Investment Strategies with
Deep Reinforcement Learning**

by

Mia GAUCI

Supervised by Dr. Kenneth Scerri
Co-supervisor Dr. Liam Butler

A synopsis submitted in partial
fulfilment of the requirements of the
award of

Bachelor of Engineering (Hons.) of the University of Malta

May 2025

Abstract- In this research, optimal investment strategies were implemented using Deep Reinforcement Learning (DRL), with a focus on the Proximal Policy Optimisation (PPO) algorithm. A portfolio of seven technology stocks with varying volatility profiles was used to construct a PPO framework for multi-stock portfolio management. PPO's performance was evaluated against a Buy-and-Hold strategy, with a supervised learning baseline using a Multilayer Perceptron (MLP) developed to explore input configurations and offer an additional benchmark. DRL agents incorporating MLPs, Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) architectures were trained and evaluated using two input configurations: normalised closing prices and market features. Evaluation through Monte Carlo simulations on 2023 market data revealed that the PPO agent using MLP with features achieved the most stable returns, averaging a return of 152%, while the CNN-based agent using price-only inputs reached a maximum return of 265% but with a lower average return. These results demonstrate that DRL can identify profitable investment strategies capable of outperforming passive benchmarks.

Keywords- Reinforcement Learning, Deep Reinforcement Learning, Proximal Policy Optimisation, Portfolio Optimisation

1. Introduction

The evolution of financial technology has reshaped trading by replacing decisions made by human traders with automated, data-driven systems. As traditional rule-based approaches struggle to adapt to volatile markets without human intervention, there is a growing demand for intelligent algorithms that can learn and improve over time [27]. Reinforcement Learning (RL) [1,6,19,20] addresses this need by allowing systems to adjust their strategies through continuous interaction with the market environment. Its growing role in institutional trading reflects its potential to support learning-based investment strategies, which forms the foundation of this study's focus on portfolio optimisation.

This project explores the application of Deep Reinforcement Learning (DRL) [5,11,18], with a particular focus on the Proximal Policy Optimisation (PPO) algorithm [19-23], for developing and optimising investment strategies across a portfolio of seven technology stocks. The research involves building a baseline supervised learning model, designing and implementing a custom PPO framework, and evaluating multiple neural network architectures under realistic market conditions. The study investigates whether DRL agents can learn to make informed portfolio allocation decisions with the goal of outperforming the buy-and-hold benchmark.

This synopsis is structured as follows: Section 2 covers some RL background and builds onto DRL, Section 3 describes the data used and its preprocessing, Section 4 details the methodology used to implement the DRL models, Section 5 presents the results obtained and Section 6 provides some conclusion and future work based on these results.

2. Literature Review

Machine Learning (ML) [2] is a fundamental building block of Artificial Intelligence (AI) [24], allowing systems to adapt and improve through experience by learning from data, rather than relying on explicitly written rules. ML is commonly divided into three main categories: supervised learning, which relies on labelled data; unsupervised learning, which identifies patterns in unlabeled data; and RL, which learns by trial and error. [2-4]

In RL, agents improve their decision-making by repeatedly interacting with an environment and learning from the outcomes of their actions. The objective is to develop strategies that maximise the cumulative reward over time. This learning process is described by a Markov Decision Process (MDP) [1,8,9], which characterises the environment using states, actions, transition probabilities and rewards. The agent aims to learn a policy that leads to the highest expected long-term return. Figure 1 shows the structure of interaction between the agent and environment as defined by the MDP model. [1,8,9]

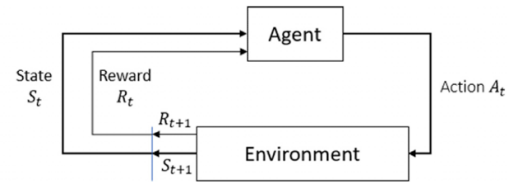


Figure 1: Visual representation of how an agent interacts with its environment under MDP framework [1].

One of the most significant developments within ML has been the emergence of Deep Learning (DL) [4], which uses deep neural networks (DNNs) [3-4] to extract meaningful features and capture non-linear patterns in data [3]. DL is particularly useful in areas that involve high-dimensional or unstructured inputs such as stock market data [3]. Building on the principles of RL, DRL integrates the DL capabilities of DNNs to extend the functionality of traditional RL methods. Classical RL relies on tabular representations of value functions and policies, which become infeasible in large or continuous environments. This problem is addressed by DRL, which enables agents to learn directly from raw input data by approximating functions using DNNs. [3-5]

Deep Reinforcement Learning approaches are primarily divided into two categories: model-based and model-free methods. [1, 5] Model-based approaches try to build a model of the environment to predict future events and plan actions in advance. Although this can lead to better learning, it often depends on accurate modelling, which may be difficult to achieve. Model-free methods learn how to make decisions by interacting with the environment directly, without relying on an internal model of how the environment operates. [5]

Model-free methods can be further categorised into value-based and policy-based approaches [1,7]. Value-based

algorithms, such as Q-Learning [6] and Deep Q-Networks [5,6], estimate the expected return of actions in given states and select those with the highest value. Policy-based approaches, on the other hand, learn a policy that directly links states to actions, frequently favouring actions associated to higher returns through gradient-based optimisation [1]. Although value-based methods are well-suited for discrete action spaces, policy-based methods are preferred in continuous settings [1,7]. Actor-critic methods combine value-based and policy-based learning within a single framework [7]. The actor selects actions according to a policy, while the critic evaluates the actions through a value function [7]. By allowing the actor to improve its decisions based on feedback from the critic, this framework promotes more consistent training. Notable Actor-Critic algorithms include Advantage Actor-Critic (A2C) [22], Deep Deterministic Policy Gradient (DDPG) [6] and Proximal Policy Optimisation (PPO) [7,18].

3. Data

Stock behaviour varies across industries, with some sectors being more volatile than others due to factors such as macroeconomic events [11]. These behavioural differences affect how the RL model must adapt to changing market conditions. To maintain a controlled setting and ensure clearer interpretation of results, the technology sector was selected due to its high trading volume and frequent price movements. The final dataset included seven large-cap technology firms namely, Apple (AAPL), Microsoft (MSFT), NVIDIA (NVDA), Alphabet (GOOGL), Amazon (AMZN), Tesla (TSLA), and Meta Platforms (META).

The required data for the selected stocks was obtained using the *yfinance* Python library, which provides access to Yahoo Finance's API [10]. Daily closing prices were chosen to provide enough detail for capturing market trends while avoiding the noise associated with intraday fluctuations. Each stock's data was stored in a separate comma-separated values (CSV) files, containing the date and respective closing prices to support structured data handling during processing.

Preprocessing of the downloaded data was needed to ensure the completeness and reliability of the input data prior to model training, as the raw data could contain missing values or anomalies. Missing entries, which typically arise from market holidays or temporary trading suspensions, were removed to prevent potential errors during the training process. Anomalies, often caused by recording inaccuracies or incomplete data points, were identified and eliminated using the modified z-score method [21] to preserve the integrity of the dataset.

In addition to the raw closing prices, four technical indicators were calculated to help the model recognise trading patterns which cannot be detected from price data alone. These were provided as an additional input, resulting in two input configurations: one using only closing prices and another combining closing prices with the indicators. The indicators

used were the Exponential Moving Average (EMA) which captures short-term trends by assigning greater weight to recent prices, the Relative Strength Index (RSI) which indicates potential overbought or oversold conditions, the Moving Average Convergence Divergence (MACD) which shows the momentum shifts and trend reversals, and Bollinger Bands which provide a measure of price volatility and range expansion [12].

To prepare the data for training, a rolling window approach was used to create an input sequence. At each time step, the state (s_t) was defined as a sequence of values from the previous days. This format allowed the agent to learn from temporally ordered inputs, enabling the model to recognise how market conditions evolve over time.

In RL, the scale of input features influences how the policy evolves over time. Large differences in stock prices can distort the learning process by giving too much weight to higher-priced inputs. This issue is visible in Figure 2, where NVDA's price movements are barely noticeable when plotted on the same scale as MSFT. To address this issue, several normalisation techniques were tested, including min-max scaling and final price normalisation [18]. Z-score standardisation [25] was selected after testing, as it preserved the variation within each asset without allowing higher-priced stocks to dominate the learning process.

All models were trained on data spanning from 1 January 2020 to 1 January 2023, a period selected due to its high volatility, including both the COVID-19 period and the subsequent tech rally [11]. A separate testing period, from 1 January 2023 to 1 January 2024, was withheld during development and reserved for final evaluation. Figure 2 shows the raw closing prices for the selected stocks, with the training and testing periods highlighted in yellow and pink, respectively.



Figure 2: Raw closing prices for the selected stocks.

4. Methodology

4.1 Benchmark Strategies

To assess whether the models were learning meaningful trading behaviour, their performance was compared against two baseline strategies. The first was a Buy-and-Hold approach, used to determine whether the agent generated value beyond a passive investment. At the beginning of each evaluation period, the portfolio was allocated equally across

the seven selected stocks and held without any further rebalancing. An initial investment of \$10,000 during the 2023 testing period grew to \$20,393, corresponding to a return of 103%.

The second benchmark was a supervised MLP [4], used to evaluate whether RL offers any real advantage over a simpler, ML approaches. The supervised MLP approach was also used to gain familiarity with the data and identify informative input feature configurations. In addition, the supervised MLP model was used to test different window sizes to see which produced the best returns. These tests showed that a 5-day window consistently performed best, and this configuration was adopted for all the RL models.

4.2 Motivation for using Proximal Policy Optimisation

As discussed in Section 2, actor-critic architectures offer several advantages over simpler RL models and were therefore selected as the focus of this investigation. Among the available actor-critic algorithms, PPO was selected based on both methodological considerations and practical implementation advantages. DDPG is an off-policy algorithm, meaning that it relies on experience replay by sampling from past interactions that may not reflect current market conditions [20]. In the context of financial trading, this can result in the agent learning from data that no longer captures relevant market dynamics. Consequently, DDPG was excluded due to the limitations associated with its learning approach. On-policy methods, on the other hand, update the policy using data collected from its most recent interactions, ensuring closer alignment with the present state of the environment. [5,14,19]

Both A2C and PPO fall under on-policy algorithms. However, PPO introduces a clipped objective function that constrains the magnitude of policy updates, which addresses a major limitation of A2C by reducing the likelihood of excessively large and destabilising changes during training [22]. This enables the policy to evolve more gradually, preserving beneficial behaviours while still adapting to new information. For these reasons, PPO was selected to investigate whether an actor-critic agent is capable of learning optimal investment strategies. [7,23]

4.3 Proximal Policy Optimisation Implementation

The RL system implemented in this project is structured as a closed feedback loop between the PPO agent and its trading environment, as illustrated in Figure 1. At each time step, the environment presents the agent with the state described in Section 3, which is then passed to the actor-critic network. The actor and critic networks share a common architecture up to their final layer. The architecture is interchangeable and different neural network types can be implemented depending on the experiment, while retaining the same PPO framework. After the shared processing, the network splits into two components. The actor outputs a mean μ_t and a

standard deviation σ_t which define the Gaussian distribution $N(\mu_t, \sigma_t)$, from which the agent samples continuous actions that determine the proportion of capital allocated to each stock. This stochastic policy framework introduces natural variability in the agent's behaviour, promoting exploration and helping to prevent premature convergence to suboptimal strategies. The critic outputs a single scalar value, denoted by $V_\phi(s_t)$, which represents the value estimated return from the current s_t . This value serves as a baseline for computing how much better or worse the chosen action was than expected during policy optimisation. After executing the action, the environment updates the agent's portfolio and cash balance and returns the next state along with a scalar reward. The reward is defined as the change in closing price from one day to the next, as this provides the agent with direct feedback on whether the chosen action was appropriate.

The training process is temporally structured, with each epoch representing a full chronological pass through the dataset. In PPO, the dataset is divided into batches of powers of 2, which is fundamental to the PPO agent, as it updates the network in batches of transitions rather than at each individual time step [23]. At each time step t , the agent receives a reward r_t and computes the Temporal Difference (TD) error, denoted by δ_t , which reflects the gap between the expected outcome and the actual reward received as given by:

$$\delta_t = r_t + \gamma V_\phi(s_t + 1) - V_\phi(s_t) \quad (1)$$

where, the discount factor $\gamma \in [0, 1]$ controls the weighting of future versus immediate rewards. A higher value promotes long-term planning, while a lower value emphasises short-term gains. In line with standard practice in financial RL, γ was set to 0.9 to balance short-term profit with long-term portfolio growth [13].

The TD errors are accumulated over each batch and used to compute the advantage estimates using the Generalised Advantage Estimation (GAE) [20] approach. GAE was chosen over simpler estimators because it strikes a balance between bias and variance [20]. GAE incorporates a smoothing parameter λ , allowing interpolation between biased low-variance and unbiased high-variance estimates. In this work, λ was set to 0.95, that helps the agent smooth out noisy financial rewards without overly dampening its ability to react to new market information [13].

The critic network learns to estimate the expected return from a given state. During training, this estimate is updated using a target value V_{target} , derived from observed rewards. The target helps minimise value loss by providing a more accurate return estimate. To compute V_{target} , the advantage at each time step t is first calculated, capturing how much better or worse the actual return was compared to the critic's baseline prediction:

$$\widehat{A}_t = \sum_{i=0}^{T-t-1} (\gamma\lambda)^i \delta_{t+i} \quad (2)$$

This advantage is then combined with the critic's current value estimate to form the target:

$$V_t^{\text{target}} = \bar{A}_t + V_\phi(s_t) \quad (3)$$

This helps the critic better approximate returns by incorporating both reward signals and the structure of the trajectory. Each trajectory consists of a sequence of states, actions, rewards, and value estimates collected over time for a given stock. Once computed, trajectories across all stocks and time steps are flattened into a single dataset, allowing standard batch Stochastic Gradient Descent (SGD) [17] to be applied during training. Random batch sampling is applied to the flattened dataset, enabling the agent to train over multiple epochs on the same batch of experiences. This exposes the networks to a varied set of transitions, reducing overfitting and helping internalise important patterns. The critic is then updated by minimising the mean squared error between its predicted state values and the computed target returns, as defined by:

$$L_{\text{critic}} = \frac{1}{T} \sum_{t=1}^T (V_\phi(s_t) - V_t^{\text{target}})^2 \quad (4)$$

where, L_{critic} denotes the loss function, measuring the mean squared error between predicted and target values across all time steps in the batch. This regression objective promotes accurate value estimates, which are critical for stable advantage calculations and, in turn, for reliable policy updates. Minimising this error helps stabilise learning by reducing noise in the policy gradients. The actor network is updated using the clipped surrogate objective given by:

$$L_{\text{actor}}(\theta) = E_t[\min(P_t(\theta)A_t, \text{clip}(P_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (5)$$

where the clipping threshold ϵ was set to 0.2, as recommended by Schulman et al. [7]. This value strikes a balance by preventing overly large policy updates that could destabilise learning, while still allowing meaningful progress in strategy improvement [18]. Rather than directly maximising the standard policy objective, PPO constrains the magnitude of each update. The probability ratio between the current and old policies is calculated as follows:

$$P_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (6)$$

This ratio measures how much the current policy π_θ deviates from the previous policy π_{θ} when selecting the same action for the same state. A value of $P_t = 1$ indicates the action has equal probability under both the old and new policies, while deviations reflect how much the new policy has shifted.

4.4 Neural Networks

Building on the actor-critic framework outlined in Section 4.3, three types of neural networks were used in this project, each selected for its suitability to different aspects of the

trading environment. MLPs were used as a fully connected baseline, offering a straightforward approach to high-dimensional function approximation [4,16]. Convolutional Neural Networks (CNN) [4,7] were included to exploit spatial patterns across rolling windows of market features [37]. Recurrent Neural Networks (RNN) [15] were chosen to model temporal dependencies in sequential price data, allowing the agent to incorporate recent trends into its decision-making process [15].

Figure 3 presents a visual diagram of the MLP-based actor-critic architecture used in this experiment. While this general structure is consistent across all models, the specific architectural components vary depending on the neural network used. The MLP consists of fully connected layers with *ReLU* [1] activations after each layer. The CNN applies instance normalization [28-29] followed by *ReLU* to capture spatial relationships in the input features. The RNN is designed to handle temporal sequences and includes an initial fully connected layer with *ReLU* activation, followed by two recurrent layers with *tanh* activations and a final layer normalisation step.

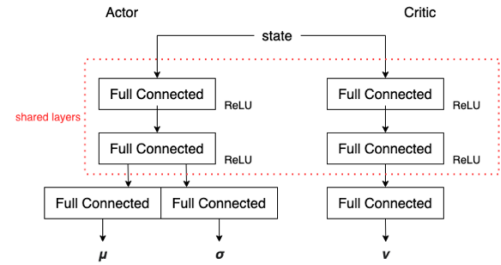


Figure 3: Architecture of the MLP Actor-Critic Network.

As outlined in Section 3, each neural network model was evaluated using two different input configurations, resulting in six distinct experimental setups. Consequently, each model required separate tuning of its architecture and training parameters, as summarised in Table 1. *L2 regularisation* [3] was applied consistently across all models, using a penalty term of 1×10^{-4} , to reduce the risk of overfitting and encourage more stable learning.

Table 1: Summary of Model Hyperparameters.

Model	Hidden Layers	Optimiser	Learning Rate	Epochs	Batch Size
MLP Closing	(16, 16, 16)	Adam	3×10^{-4}	200	32
MLP Features	(32, 32, 32)	Adam	1×10^{-4}	500	32
CNN Closing	(32, 32, 32)	AdamW	5×10^{-4}	700	64
CNN Features	(128, 64, 32)	AdamW	3×10^{-4}	500	64
RNN Closing	(32, 16, 16)	SGD	3×10^{-4}	500	128
RNN Features	(64, 32, 32)	SGD	3×10^{-4}	500	128

Figure 4 shows the training curves for the MLP model using closing price as input, with the actor loss (left) and average log-probability (right) plotted over training epochs. The actor loss, defined in (5), reflects how the policy evolves during training, with a gradual decline indicating it has converged. The average log-probability provides a measure of how much the updated policy diverges from its previous form, showing whether the policy managed to stabilise or not.

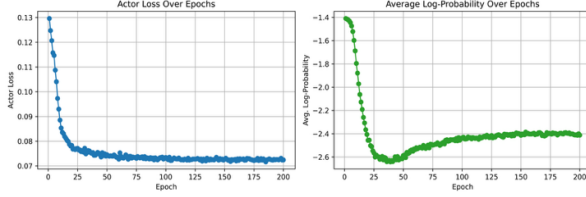


Figure 4: MLP closing Actor Loss (left) and average log-probability (right).

Similar plots were generated for each model and input configurations, capturing the dynamics of training under different architectural and data conditions. While the full set of visualisations is included in the main dissertation, the results are summarised in Table 2, which presents the approximate epoch at which the actor loss converged, the corresponding final actor loss value the epoch at which the policy stabilised, and the final average log-probability observed for each experiment.

Table 2: PPO Convergence and Stability Metrics.

Model	Actor Loss Converged	Final Actor Loss	Policy Stabilised	Final Avg Log-Prob
MLP Closing	~75	0.072	~125	-2.4
MLP Features	~200	0.087	~100	-5.0
CNN Closing	~100	0.055	~300	-2.0
CNN Features	~300	0.015	~400	-1.80
RNN Closing	~25	0.098	~300	-15.5
RNN Features	~100	0.050	>500	-2700

The environment developed for this project is a custom market simulation tailored to the PPO agent. The environment replicates a basic portfolio management workflow where at each time step, the agent observes recent price data, allocates capital across assets, and receives a reward based on the resulting change in portfolio value. Two core functions define its logic, *reset()* and *step()*. The *reset()* function reinitialises the environment, setting the cash balance to \$10,000, clearing holdings, and providing the initial state. The *step()* function executes the agent's actions by liquidating assets with negative signals and reallocating available cash proportionally across positively weighted stocks, ensuring a consistent and controlled learning process throughout training.

With the PPO agent, neural networks, and custom environment in place, the complete trading system was assembled. The simulation runs over historical data day by day, allowing the agent to make trading decisions and update its portfolio accordingly. Figure 5 shows that, despite identical model weights, portfolio outcomes vary substantially due to the stochastic nature of the PPO policy. The plot displays the median trajectory alongside an approximate ± 1 standard deviation band which is the range between the 40th and 160th trajectories out of 200 Monte Carlo runs. This variation arises because, as discussed in Section 4.3, PPO samples its actions from a Gaussian

distribution rather than following a fixed rule. As a result, even with the same inputs and model weights, the agent may make slightly different decisions each time it runs. These small differences accumulate, and in a non-linear, path-dependent environment such as financial markets, they can lead to significantly different portfolio outcomes over time. While the environment is deterministic, the randomness introduced through action sampling drives this difference.

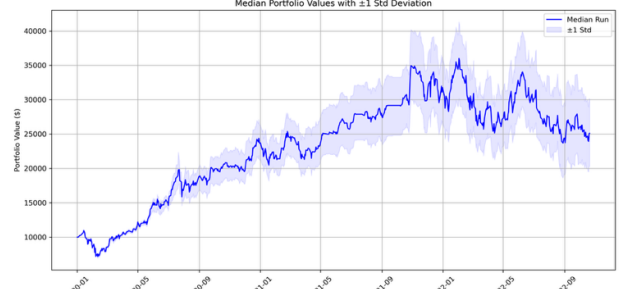


Figure 5: Median Portfolio Value with ± 1 standard deviation.

5. Results and Discussion

To capture this variability, each trained agent was evaluated across 200 independent rollouts on both the training and test data using a Monte Carlo approach, to assess both the average performance and the model's sensitivity to random initialisation. The resulting portfolio values were used to generate violin plots, with Figure 6 presenting the test set results while an equivalent plot for the training data is included in the full dissertation.

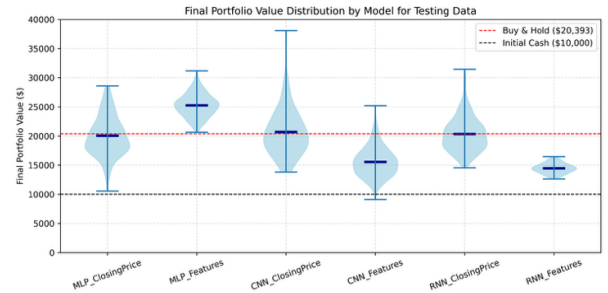


Figure 6: Violin Plot for 200 Test Rollouts per Model.

The MLP Closing Price model demonstrates moderate performance on the test set. Its violin plot is relatively tall and narrow, with most outcomes concentrated just below the buy-and-hold benchmark. Approximately two-fifths of the runs outperform the benchmark, and all runs maintain portfolio values above the initial capital, indicating consistent capital preservation. Although the restricted input may have limited the model's ability to fully benefit from the 2023 upward trend, the results reflect stable behaviour. In contrast, the MLP Features model performs strongly across all test rollouts. Every run exceeds the benchmark, with an average return approximately 50 percent higher and the lowest-performing run still yielding a clear gain. The compact and symmetric

distribution, with no extreme values, suggests a well-learned policy that performs reliably across different conditions.

The violin plot for the CNN Closing model is relatively wide, with results distributed evenly around the buy-and-hold benchmark. Although the average return is only marginally higher than the benchmark, the upper tail extends notably further, with the best-performing run exceeding the highest return observed in the MLP Features model. This indicates the model's potential for substantial gains under favourable conditions. However, the broader spread and variability across runs reflect inconsistent learning, suggesting the model occasionally captures useful signals but lacks the reliability of more stable alternatives. In comparison, the CNN Features model displays a similarly wide distribution, with most values clustering just above the initial capital. The model rarely exceeds the benchmark and is the only configuration to produce final portfolio values below the initial investment. These outcomes suggest that the additional input features may hinder rather than help the CNN architecture, leading to overfitting and unstable policy behaviour in high-dimensional environments. This is consistent with the findings in Table 2, where the CNN Features model took the longest to converge and exhibited an average final log-probability of approximately -180, indicating poor policy confidence.

The violin plot for the RNN Closing model is relatively compact, with most values clustered near the buy-and-hold benchmark. Its average return closely aligns with the MLP and CNN Closing models, though with less variability and only one notable outlier. This suggests that while the model may not capture exceptional gains, it tends to produce more consistent outcomes. The architecture appears to have picked up on short-term patterns, which likely contributed to its steady performance in a predominantly upward-trending market. The RNN Features model, by contrast, exhibits an extremely narrow distribution centered well below the benchmark. Although it preserves capital across all runs, it fails to outperform in any instance. This is evidenced in Table 2, where the RNN Features model failed to stabilise, indicating highly uncertain policy behaviour.

Based on the testing distribution of final portfolio values in Figure 6, each model can be classified by its relative risk and reward profile, as summarised in Table 3. These classifications reflect return volatility and stability, aligning with practical investment trade-offs where some investors prioritise stability and moderate gains, while others seek higher returns despite increased risk. Notably, the MLP Features model stands out for consistent outperformance of the buy-and-hold baseline, making it ideal for risk-averse strategies, whereas the CNN Closing Price model, despite its higher variability, delivered some of the highest returns and is thus suited to higher-risk preferences seeking greater rewards.

Table 3: PPO Risk-Reward Classification.

Model	Risk Level	Reward Level
MLP Closing	Moderate	Moderate
MLP Features	Low	Moderate
CNN Closing	High	High
CNN Features	Moderate	Low
RNN Closing	Moderate	Moderate
RNN Features	Low	Low

The training time is an important practical factor alongside model performance. Table 4 shows that MLP models train much faster than CNNs and RNNs, partly due to lighter preprocessing and smaller inputs. The MLP Features model offers strong, consistent returns with minimal training time, making it well suited for practical use. While the CNN Closing model can achieve higher returns, its long training time and variable outcomes reduce its suitability for time-sensitive applications.

Table 4: Training Duration by Model

Model	Epochs	Total Time	Time/Epoch (min)
MLP Closing	200	03:55	1.18
MLP Features	500	05:40	0.68
CNN Closing	700	33:49:00	2.90
CNN Features	500	44:09:00	5.29
RNN Closing	500	38:42:00	4.64
RNN Features	500	51:21:49	6.16

To conclude the analysis, Figure 7 compares the median portfolio values of the feature-based PPO models from Figure 6 against the supervised MLP baseline and the buy-and-hold strategy. The MLP Features model clearly leads, showing steady growth and ending the year with a return of around 155%, more than double that of the supervised MLP. The CNN Features model remains competitive, generally tracking the baseline with more volatility but finishing slightly ahead. In contrast, the RNN Features model underperforms throughout, plateauing early and reinforcing its earlier weak results. This suggests that RL, when coupled with appropriately chosen architectures, can better exploit sequential feedback in volatile markets.

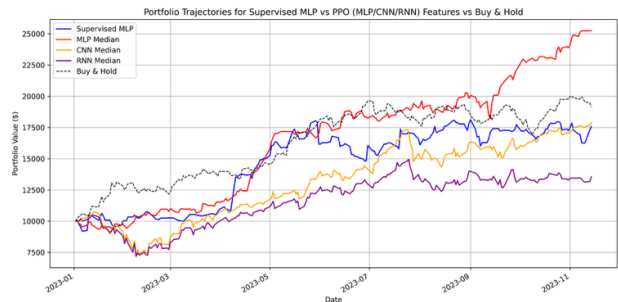


Figure 7: Median PPO Trajectories for MLP, CNN, RNN vs Supervised MLP and Buy & Hold on 2023 Test Data

6. Conclusion

This project explored the use of DRL, specifically PPO, for portfolio management across multiple stocks. A custom trading environment was developed to evaluate PPO agents built on MLP, CNN, and RNN architectures with different input configurations. Among these, the MLP model trained on engineered market features achieved the strongest and most consistent performance, outperforming both the buy-and-hold benchmark and a supervised MLP baseline with an average return exceeding 150%. While CNN models achieved some of the highest individual returns, their performance varied significantly. RNN models did not perform well, struggling to retain relevant patterns across the time series. Overall, the results show that PPO, when paired with the right model and inputs, can support the development of profitable trading strategies.

Future work could explore the use of higher-frequency market data, such as hourly or minutely prices, to better capture short-term trends and improve the agent's responsiveness to intra-day movements. The PPO agent could also benefit from incorporating more sophisticated reward structures, such as risk-adjusted returns, drawdown penalties, or Sharpe ratios, which could better align the agent's behaviour with different investment goals. Expanding the framework to other market sectors would allow for broader validation under varying conditions. Additionally, tailoring input features to suit each architecture, for example, reducing noise in CNN inputs or enriching MLPs with sentiment data could improve performance. Finally, integrating PPO's entropy bonus and tuning its parameters may enhance exploration during early training, helping the agent avoid premature convergence.

Future research could also focus on improving model performance by replacing the basic RNNs with more advanced architectures such as LSTM [15-16], which are better suited for capturing temporal dependencies in financial data. Standardising the environment using established libraries such as Gymnasium [30] would enhance compatibility and streamline integration with other RL tools. Exploring alternative actor-critic algorithms beyond PPO, both on-policy and off-policy, would also provide valuable insights into their relative strengths under identical conditions [14,22]. In the long term, connecting the agent to real-time market data through broker APIs such as Interactive Brokers would allow for a more realistic evaluation, incorporating factors such as latency, slippage, and liquidity constraints.

Although financial markets are inherently unpredictable, this study shows that DRL offers a structured, adaptive approach to managing that uncertainty. When paired with suitable architectures and input features, DRL can outperform both passive benchmarks and supervised learning models. With continued development, DRL has the potential to outperform the best traders in the industry.

References

- [1] R.S. Sutton, A.G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, Cambridge, 1998.
- [2] N. Xie, W. Zhang, Empowering big data analytics through machine learning: Applications, challenges, and future directions, *Appl. Comput. Eng.* 1 (2023), pp. 16–24.
- [3] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, Cambridge, 2016.
- [4] J. Schmidhuber, Deep learning in neural networks: An overview, *Neural Netw.* 61 (2015), pp. 85–117.
- [5] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, et al., Human-level control through deep reinforcement learning, *Nature* 518 (2015), pp. 529–533.
- [6] S. Dridi, *Reinforcement Learning – A Systematic Literature Review*, OSF Preprints (2024), pp. 1–25.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal Policy Optimization Algorithms, *arXiv preprint arXiv:1707.06347* (2017), pp. 1–12.
- [8] Feinberg, E. A. and Shwartz, A. *Handbook of Markov Decision Processes: Methods and Applications*. Springer, Boston, MA, 2012.
- [9] Puterman, M. L. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 2014.
- [10] Yahoo Finance, Yahoo Finance – Stock Market Live, Quotes, Business & Finance News. [Online] <https://finance.yahoo.com/>. [Cited: May 10, 2025.]
- [11] Z. Zhang, S. Zohren, S. Roberts, Deep Reinforcement Learning for Trading, *J. Financ. Data Sci.* 2 (2020), pp. 25–40.
- [12] J.J. Murphy, *Technical Analysis of the Financial Markets*, New York Institute of Finance, New York, 1999.
- [13] J. Doe, A. Smith, Strategic discounting of future rewards in financial reinforcement learning, *J. Financ. AI Res.* 12 (2023), pp. 45–60.
- [14] I. Kostrikov, D. Yarats, R. Fergus, What matters in on-policy reinforcement learning? A large-scale empirical study, *Proc. Int. Conf. Learn. Represent.* (2020), pp. 1–12.
- [15] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, Cambridge, 2016.
- [16] S. Hochreiter, J. Schmidhuber, Long Short-Term Memory, *Neural Computation*, MIT Press, Cambridge, 1997.
- [17] R.S. Sutton, D.A. McAllester, S.P. Singh, Y. Mansour, Policy Gradient Methods for Reinforcement Learning with Function Approximation, *Adv. Neural Inf. Process. Syst.* 12 (2000), pp. 1057–1063.
- [18] Z. Liang, H. Chen, J. Zhu, K. Jiang, Y. Li, Adversarial Deep Reinforcement Learning in Portfolio Management, *arXiv preprint arXiv:1808.09940* (2018), pp. 1–10.
- [19] L. Graesser, W.L. Keng, *Foundations of Deep Reinforcement Learning*, Addison-Wesley, Boston, 2019.
- [20] M. Lapan, *Deep Reinforcement Learning Hands-On*, second ed., Packt Publishing, Birmingham, 2020.
- [21] B. Iglewicz, D.C. Hoaglin, *How to Detect and Handle Outliers*, ASQC Quality Press, Milwaukee, 1993.
- [22] H.-C. Jang, Y.-C. Huang, H.-A. Chiu, A Study on the Effectiveness of A2C and A3C Reinforcement Learning in Parking Space Search in Urban Areas Problem, *Int. Conf. Inf. Commun. Technol. Converg.* (2020), pp. 567–571.
- [23] Y. Sun, X. Yuan, W. Liu, C. Sun, Model-Based Reinforcement Learning via Proximal Policy Optimization, *Chin. Autom. Congr.* (2019), pp. 4736–4740.
- [24] S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, fourth ed., Pearson, Harlow, 2020.
- [25] C.M. Bishop, *Pattern Recognition and Machine Learning*, Springer, New York, 2006.
- [26] Z. Zhang, S. Zohren, S. Roberts, Deep Reinforcement Learning for Trading, *J. Financ. Data Sci.* 2 (2020), pp. 25–40.
- [27] D. Ulyanov, A. Vedaldi, V. Lempitsky, Instance Normalization: The Missing Ingredient for Fast Stylization, *arXiv preprint arXiv:1607.08022* (2016), pp. 1–6.
- [28] I. Loshchilov, F. Hutter, Decoupled Weight Decay Regularization, *arXiv preprint arXiv:1711.05101* (2017), pp. 1–8.
- [29] The Farama Foundation, Gymnasium: A Standard API for Reinforcement Learning Environments. [Online] <https://github.com/Farama-Foundation/Gymnasium>. [Cited: May 10, 2025.]